

# Network Intrusion Detection System (NIDS) — Setup Guide

This guide builds a working NIDS lab using **Suricata** (the modern successor to Snort, actively maintained by the OISF, and largely rule-compatible with Snort's syntax). Snort-specific notes are called out where it differs. Everything here targets **Ubuntu/Debian**; adjust package manager commands for other distros.

**Scope note:** This is defensive infrastructure — detecting and alerting on attacks against *your own* network. Nothing here helps craft exploits; it helps you see them coming.

## 1. Set Up the IDS

### 1.1 Choose your deployment mode

| Mode                 | What it does                               | Where it sits                                         |
|----------------------|--------------------------------------------|-------------------------------------------------------|
| <b>IDS (passive)</b> | Monitors a copy of traffic, alerts only    | Connected to a SPAN/mirror port or TAP                |
| <b>IPS (inline)</b>  | Sits in the traffic path, can drop packets | Between router and network (NFQUEUE/AF_PACKET inline) |

Start in IDS mode — get visibility and tune rules before you let anything auto-block live traffic.

### 1.2 Install Suricata

```
# Ubuntu/Debian
sudo add-apt-repository ppa:oisf/suricata-stable -y
sudo apt update
sudo apt install suricata -y

# Confirm install and version
suricata --build-info
suricata -V
```

Snort alternative:

```
sudo apt install snort -y
```

### 1.3 Identify your monitoring interface

```
ip -br addr show
```

Pick the interface that sees the traffic you want to inspect (e.g. eth0, or a SPAN port interface, or the WAN-facing NIC on a router/firewall box).

### 1.4 Basic configuration

Edit /etc/suricata/suricata.yaml:

```
# Define your protected network(s) – critical for rule accuracy
```

```

vars:
  address-groups:
    HOME_NET: "[192.168.1.0/24]"      # your internal network
    EXTERNAL_NET: " !$HOME_NET"

# Set the interface to monitor
af-packet:
  - interface: eth0
    cluster-id: 99
    cluster-type: cluster_flow
    defrag: yes

# Where alerts/logs go
default-log-dir: /var/log/suricata/

outputs:
  - eve-log:
    enabled: yes
    filetype: regular
    filename: eve.json
    types:
      - alert
      - http
      - dns
      - tls
      - ssh
      - flow
      - stats
  - fast:
    enabled: yes
    filename: fast.log

```

HOME\_NET matters more than almost any other setting — most rules reference it, and a wrong value causes both missed detections and false positives.

Validate the config:

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
```

---

## 2. Configure Rules and Alerts

### 2.1 Rule sources

Suricata ships with a rule manager, `suricata-update`, that pulls curated rulesets (Emerging Threats Open, ET Pro if licensed, Abuse.ch feeds, etc.):

```

sudo suricata-update
sudo suricata-update list-sources      # see available rule
feeds
sudo suricata-update enable-source et/open # Emerging Threats Open
(free, high quality)
sudo suricata-update

```

This populates `/var/lib/suricata/rules/suricata.rules`. Point the config at it:

```

default-rule-path: /var/lib/suricata/rules
rule-files:
  - suricata.rules

```

### 2.2 Writing custom rules

Custom rules live in a separate file,  
e.g. /etc/suricata/rules/local.rules:

```
# Detect ICMP ping sweeps (reconnaissance)
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected";
itype:8; sid:1000001; rev:1;)

# Detect SSH brute-force attempts (>5 connection attempts in 60s
from one source)
alert tcp any any -> $HOME_NET 22 (msg:"Possible SSH Brute Force";
flow:to_server; \
  flags:S; threshold:type threshold, track by_src, count 5, seconds
60; sid:1000002; rev:1;)

# Detect suspicious outbound traffic to known bad ports (e.g. common
C2 ports)
alert tcp $HOME_NET any -> any 4444 (msg:"Possible Metasploit/C2
Callback"; sid:1000003; rev:1;)

# Detect port scanning behavior
alert tcp any any -> $HOME_NET any (msg:"Possible Port Scan
Detected"; flags:S; \
  threshold:type threshold, track by_src, count 20, seconds 10;
sid:1000004; rev:1;)

# Detect plaintext credential-looking strings in HTTP traffic
alert http any any -> $HOME_NET any (msg:"Plaintext Password in HTTP
Request"; \
  content:"password="; http_uri; sid:1000005; rev:1;)

# Detect SQL injection attempt patterns
alert http any any -> $HOME_NET any (msg:"Possible SQL Injection
Attempt"; \
  content:"UNION SELECT"; nocase; http_uri; sid:1000006; rev:1;)
```

**Rule anatomy** (Suricata/Snort share this syntax):

```
action proto src_ip src_port -> dst_ip dst_port (options)
```

- action: alert, drop (IPS mode), pass, reject
- options: msg (alert text), sid (unique rule ID — keep local rules  $\geq 1,000,000$ ), rev (revision), detection content (content, pcre, flags, threshold, etc.)

Register your custom file:

```
rule-files:
- suricata.rules
- local.rules
```

## 2.3 Alert severity and classification

Add classification metadata so alerts can be triaged by priority:

```
alert tcp any any -> $HOME_NET 22 (msg:"SSH Brute Force"; ...;
classtype:attempted-recon; priority:2; sid:1000002; rev:1;)
```

/etc/suricata/classification.config maps classtype values to severity levels — useful once you're feeding alerts into a SIEM or dashboard.

## 2.4 Test rule syntax before deploying

```
sudo suricata -T -c /etc/suricata/suricata.yaml -S
/etc/suricata/rules/local.rules -v
```

---

## 3. Continuous Monitoring

### 3.1 Run as a systemd service (persistent monitoring)

```
sudo systemctl enable suricata
sudo systemctl start suricata
sudo systemctl status suricata
```

### 3.2 Live-tail alerts

```
# Human-readable fast log
sudo tail -f /var/log/suricata/fast.log

# Structured JSON (best for parsing/dashboards)
sudo tail -f /var/log/suricata/eve.json | jq
'select(.event_type=="alert")'
```

### 3.3 Automated log rotation

logrotate config at /etc/logrotate.d/suricata (usually installed by default) prevents eve.json from growing unbounded:

```
/var/log/suricata/*.log /var/log/suricata/*.json {
    daily
    rotate 14
    missingok
    compress
    delaycompress
    postrotate
        /bin/kill -HUP $(cat /var/run/suricata.pid 2>/dev/null)
2>/dev/null || true
    endscript
}
```

### 3.4 Keep rules current

Automate suricata-update via cron so new threat signatures arrive automatically:

```
sudo crontab -e
# Add:
0 3 * * * /usr/bin/suricata-update && systemctl reload suricata
```

### 3.5 Health/performance monitoring

Watch for dropped packets (a sign your box can't keep up with traffic volume):

```
sudo tail -f /var/log/suricata/stats.log
```

Key fields: capture.kernel\_drops — if this climbs, you need a faster NIC config (AF\_PACKET tuning, multiple threads) or more CPU.

---

## 4. Response Mechanisms

### 4.1 Passive: alert-only pipeline (safest starting point)

Alerts land in eve.json → forwarded to a SIEM/log system → analyst reviews and manually responds. No auto-blocking risk of false-positive outages.

## 4.2 Active: automated blocking with a companion script

A common pattern is to have a script watch `eve.json` for high-confidence alerts and add firewall blocks:

```
#!/bin/bash
# /usr/local/bin/auto-block.sh
# Watches Suricata alerts and blocks offending IPs via iptables

tail -Fn0 /var/log/suricata/eve.json | while read -r line; do
    event_type=$(echo "$line" | jq -r '.event_type')
    if [ "$event_type" == "alert" ]; then
        sig=$(echo "$line" | jq -r '.alert.signature')
        src_ip=$(echo "$line" | jq -r '.src_ip')
        # Only auto-block on specific high-confidence sids – don't
        # blanket-block on everything
        sid=$(echo "$line" | jq -r '.alert.signature_id')
        if [[ "$sid" == "1000002" || "$sid" == "1000004" ]]; then
            if ! sudo iptables -C INPUT -s "$src_ip" -j DROP 2>/dev/null;
            then
                sudo iptables -A INPUT -s "$src_ip" -j DROP
                echo "$(date): Blocked $src_ip for '$sig' (sid $sid)" >>
                /var/log/auto-block.log
            fi
        fi
    fi
done
```

Run it as a service:

```
sudo chmod +x /usr/local/bin/auto-block.sh
sudo nohup /usr/local/bin/auto-block.sh &
```

For a more mature version of this exact pattern, look at **fail2ban** (log-watching + banning framework) — it can be pointed at Suricata's `eve.json` instead of writing this from scratch.

## 4.3 True inline IPS mode (drop packets in real time)

Switch Suricata to NFQUEUE inline mode so drop rules actually block traffic before it reaches the host:

```
# Route traffic through NFQUEUE
sudo iptables -I FORWARD -j NFQUEUE --queue-num 0

# Run Suricata inline
sudo suricata -c /etc/suricata/suricata.yaml -q 0
```

Then change rule actions from alert to drop for the specific signatures you trust enough to auto-enforce.

## 4.4 Notifications

Pipe alerts to email/Slack/webhook so responders aren't only relying on log-tailing:

```
# Example: simple email alert on any new alert line
tail -Fn0 /var/log/suricata/fast.log | while read -r line; do
    echo "$line" | mail -s "Suricata Alert" you@example.com
done
```

For production, prefer routing through a SIEM's alerting rules rather than raw shell pipes.

---

## 5. Visualization (Optional)

### Option A — ELK Stack (Elasticsearch + Logstash/Filebeat + Kibana)

The most common pairing for Suricata:

```
# Filebeat ships eve.json into Elasticsearch
sudo apt install filebeat
sudo filebeat modules enable suricata
sudo filebeat setup
sudo systemctl start filebeat
```

Kibana then gives you dashboards over alert volume, top talkers, signature frequency, geomap of source IPs, etc. Elastic publishes a ready-made Suricata module/dashboard set — enabling it via `filebeat modules enable suricata` gets you most of the way with no manual dashboard building.

### Option B — EveBox

A lightweight, purpose-built alert viewer for Suricata's `eve.json` — much less setup than ELK if you just want a browsable alert console:

```
# Debian/Ubuntu package available from EveBox's repo
sudo apt install evebox
sudo evebox oneshot /var/log/suricata/eve.json # quick local
viewer
```

### Option C — Security Onion

A full pre-built distro that bundles Suricata/Zeek + Elastic stack + Kibana dashboards out of the box — worth considering if you'd rather not hand-assemble the pipeline above.

---

## Suggested Lab Workflow

1. Install Suricata, set `HOME_NET` correctly, validate config.
  2. Enable ET Open ruleset + add 4-6 custom rules (port scan, brute force, C2 port, SQLi pattern).
  3. Run in IDS mode for a few days; tail `eve.json`, tune out false positives (adjust threshold, refine content matches).
  4. Add the auto-block script for 1-2 high-confidence sids only.
  5. Stand up EveBox or Filebeat→Kibana for a visual dashboard.
  6. Generate test traffic safely with tools like `nmap` **against your own lab host** to confirm rules fire (e.g. `nmap -sS <lab-target>` should trip the port-scan rule).
- 

### Reference commands cheat-sheet

```
sudo systemctl status suricata # service status
sudo suricata -T -c /etc/suricata/suricata.yaml -v # test config
sudo suricata-update # refresh rules
sudo tail -f /var/log/suricata/fast.log # live alerts
sudo tail -f /var/log/suricata/eve.json | jq
'select(.event_type=="alert")'
```